



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Code Formatter and Indenter Tool

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements

for the Course of

CSA1403 – COMPILER DESIGN

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

AIDS

Submitted by

ARUNPRASATH R (192524077)

KAILASH V (192521232)

Under the Supervision of

Dr. G. Michael

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “Code Formatter and Indenter Tool” has been carried out by **ARUNPRASATH R (192524077)** and KAILASH V(192521232) under the supervision of **Dr. G. Michael** and is submitted in partial fulfilment of the requirements for the current semester of the **B. Tech AIDS** program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. DEVIPRIYA
Professor
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY

Dr.G.Michael
Professor
SIMATS Engineering,
SIMATS, Chennai – 602105.



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DECLARATION

We, Arunprasath R (192524077) and Kailash V (192521232) of the Department of AIDS & IT, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “CodeAlign: Automatic Source Code Formatter” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: 07/09/2026

Signature of the Students with Names
ARUNPRASATH R

Individual Contribution Statement

(Mandatory for all team projects)

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Arunprasath R (192524077)	Front-end interface design, lexical analysis module, integration of formatting engine with UI	Front-end development (HTML, CSS, JavaScript); token identification logic	Functional testing of formatting scenarios (TC01–TC05)	The content research theory and output execution	50 %
Kailash V (192521232)	Back-end development, structural analysis module, Flask integration, documentation	Back-end development (Python, Flask); structural analysis and indentation logic	Interface and edge-case testing (TC06–TC07)	The figure and table representation with their suitable points	50 %
Total					100%

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

ABSTRACT

CodeAlign is a web-based source code formatting and alignment tool designed to automatically improve the readability, structure, and consistency of source programs. Manual formatting of large or deeply nested programs is repetitive, time-consuming, and prone to inconsistency, which reduces readability and increases maintenance and debugging effort — a problem that is significant both for practising developers and for students trying to connect compiler theory with real code.

To address this, CodeAlign analyses input source code using fundamental compiler design concepts — lexical analysis, token identification, syntax-oriented structural analysis, and rulebased code generation — to identify programming constructs, indentation levels, brackets, operators, keywords, and other syntactic elements, and then reconstructs the code in a properly formatted form.

The system was implemented as a web application with an HTML/CSS/JavaScript front-end and a Python (Flask) back-end that performs the token identification, structural analysis, and formatting logic. Users paste or upload source code into a browser-based editor and receive instantly formatted output through a Format Code action.

The implemented system was verified against seven representative test cases covering simple statements, conditional and nested blocks, operator spacing, multi-statement input, empty input, and interface theme switching; all test cases passed, confirming that CodeAlign consistently improves indentation, spacing, and block alignment without altering program logic. The project demonstrates that compiler design principles extend naturally beyond traditional compiler construction into practical developer tooling, and it provides a working foundation for future extensions such as multi-language support, AST-based parsing, syntax error detection, and customizable formatting rules.

Keywords: Compiler Design, Source Code Formatting, Lexical Analysis, Syntax Analysis, Web Application, Flask.

TABLE OF CONTENTS

Sl. No	Title	Page No
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction	9
2	CHAPTER 2: Literature Review	12
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	14
4	CHAPTER 4: Implementation, Testing & Results, Project Management	19
5	CHAPTER 5: Conclusion & Reflection	23
	References	25
	Appendices	26

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1	OUTPUT FRONT PAGE	10
2	INPUT ENTERING FIELD	12
3	OUTPUT FIELD	17

LIST OF TABLES

Table No.	Table Name	Page No.
-	Individual Contribution Statement	4
2.1	Comparative Analysis of Existing Approaches	12
3.1	Stakeholder / User Requirements	14
3.2	Functional & Non-Functional Requirements	14
3.3	Constraints & Applicable Standards	14
3.4	Design Specifications	15
3.5	Alternative Solutions – Evaluation Matrix	15
3.6	Trade-off Analysis	17
3.7	Sustainability, Ethics, Safety & Security Considerations	17
3.8	Risk Register	18
4.1	Software Tools Used	19

4.2	Test Plan & Verification	19
4.3	Test Results	20
4.4	Comparison with Existing Solutions & Achievement of Objectives	21
4.5	Roles & Contribution & Budget	22
-	Capstone Project Outcome Mapping Sheet	27

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
HTML	Hyper Text Markup Language	-
CSS	Cascading Style Sheets	-
JS	JavaScript	-
IDE	Integrated Development Environment	-
AST	Abstract Syntax Tree	-
API	Application Programming Interface	-
HTTP	HyperText Transfer Protocol	-
UI	User Interface	-
TC	Test Case	-
SO	Student Outcome	-
%	Percentage	-
s	Seconds	sec

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

In modern software development, source code is frequently written, modified, and maintained by multiple developers. Proper formatting and indentation are essential for making source code readable, understandable, and maintainable. However, manually formatting source code can be time-consuming, especially when working with large programs or complex nested structures.

CodeAlign – Automatic Source Code Formatter is a web-based application developed to address this problem by automatically analyzing and formatting source code. The system accepts source code as input and produces a properly structured and consistently indented version as output, helping developers reduce manual formatting effort and improving the overall readability of programs.

The project is developed based on fundamental Compiler Design concepts. Compiler design involves several phases, including lexical analysis, syntax analysis, semantic analysis, intermediate representation, optimization, and code generation. CodeAlign primarily makes use of concepts related to lexical analysis, token identification, syntax analysis, and structural analysis to understand the organization of source code and apply suitable formatting rules.

1.2 Need for the Project

Writing correctly formatted code is important for understanding program structure and maintaining software efficiently. Inconsistent indentation, improper spacing, and poorly organized blocks make programs difficult to read and debug, and this problem affects individual developers, student programmers, and teams collaborating on shared source code alike.

Although modern Integrated Development Environments (IDEs) and code editors provide formatting features, they largely depend on language-specific formatting engines and predefined configurations that are opaque to the end user. For an academic project grounded in Compiler Design, there is also a need to demonstrate — rather than merely use — how the internal structure of source code can be analyzed and used to drive formatting decisions. This is the engineering significance of the project: it makes the compiler-theory pipeline (tokens → structure → rules → output) visible and reusable in a lightweight, accessible tool.

1.3 Problem Statement

Manual source code formatting requires continuous attention to indentation, brackets, spacing, and programming constructs. Incorrect or inconsistent formatting reduces

readability and makes identifying logical or structural problems more difficult, particularly as programs grow in size and nesting depth. The problem addressed by this project is to develop a system that can automatically analyze the structure of source code and apply appropriate formatting and indentation rules, thereby reducing manual effort and improving code readability, while remaining simple enough to run entirely through a browser-based interface.

1.4 Project Aim

To design and implement a web-based automatic source code formatting system that applies compiler design principles — lexical analysis and structural analysis — to consistently and accurately reformat source code without altering its logical behaviour.

1.5 Project Objectives

- To develop a web-based automatic source code formatting system.
- To analyze source code using fundamental compiler design concepts.
- To identify important lexical elements such as keywords, operators, identifiers, and delimiters.
- To understand and correctly process nested blocks and program structures.
- To automatically apply appropriate indentation and formatting rules.
- To improve source code readability and consistency.
- To provide a simple and user-friendly interface for entering and formatting code.
- To evaluate the system through representative test cases covering statements, conditionals, nesting, and edge cases.

1.6 Scope

Included: analysis and formatting of source code through a web-based interface; token identification (keywords, identifiers, operators, literals, delimiters); structural analysis of blocks and nesting; automatic indentation, spacing, and bracket alignment; a responsive frontend editor and output viewer; a Flask-based back-end that performs the formatting logic; and Python-focused formatting as the initial supported language (Black-style conventions). Excluded (in the current version): full compilation or execution of the submitted code; multilanguage formatting support beyond the initial target language; generation of a complete Abstract Syntax Tree; and semantic-level error detection.

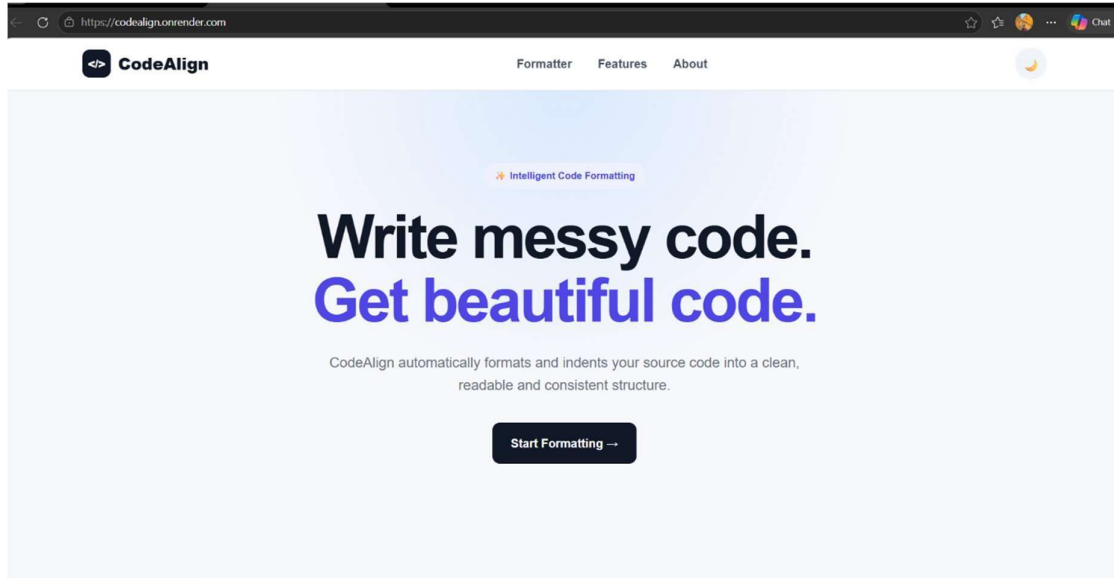
Assumptions: the input is syntactically well-formed source code in the supported language; users have basic familiarity with the language they are formatting; and the tool is used as a formatting aid rather than a full IDE replacement.

Boundary conditions: extremely large files may increase processing time; the current formatting engine applies a fixed, non-customizable rule set; and no persistent storage of usersubmitted code is implemented.

1.7 Expected Engineering Outcome

The expected outcome is a working web-based software system (an interactive process/tool rather than a physical device) comprising: a browser-based code editor and output interface; a lexical and structural analysis pipeline; and a rule-based formatting engine served through a Flask back-end — collectively demonstrating the practical application of compiler design concepts in a deployable developer tool.

Fig 1: OUTPUT FRONT PAGE



CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature and existing technologies were identified through:

(i) foundational compiler-design textbooks covering lexical and syntax analysis; (ii) official documentation of established source-code formatters (Clang-Format, Prettier, Black); and (iii) review of IDE-integrated formatting features commonly used by developers. The review focused on how each source handles token identification, structural/nesting analysis, and rule-based output generation, since these are the same stages implemented in CodeAlign.

2.2 Review of Existing Work

Aho, Lam, Sethi, and Ullman’s *Compilers: Principles, Techniques, and Tools* and Cooper and Torczon’s *Engineering a Compiler* establish the standard compiler pipeline — lexical analysis, syntax analysis, and structural/semantic processing — that this project adapts for a formatting (rather than compilation) purpose.

Clang-Format is a mature, configurable command-line formatter for C-family languages, driven by style files that define indentation, spacing, and bracket rules. Prettier takes an opinionated, low-configuration approach and supports multiple languages and editor integrations through a documented API. Black is the de-facto standard formatter for Python, producing deterministic output and validating results against the Python Abstract Syntax Tree. Each of these tools is mature, widely deployed, and effective, but each also functions largely as a black box to the end user — the internal tokenization and structural-analysis logic is not exposed as part of the user experience.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Manual formatting (developer discipline)	No tooling dependency; full control over style	Time-consuming, inconsistent, errorprone on large/nested files	Motivates automation — the core problem CodeAlign addresses
IDE auto-formatters (e.g., VS Code, PyCharm built-in formatting)	Integrated into the editor; fast, familiar workflow	Formatting logic is opaque; tied to a specific editor/environment	CodeAlign externalizes the same idea as a standalone, editor-independent web tool

Prettier	Opinionated, consistent output; broad	Abstracts away the underlying	CodeAlign intentionally exposes the tokenization/structural-
	language/plugin support; simple CLI/API	parsing/AST logic from the end user	analysis stages to fulfil its academic (compiler-design) purpose
Black (Python formatter)	Deterministic, ASTchecked formatting; defacto standard for Python	Python-specific; not web-based; requires local installation	CodeAlign adopts Black-style conventions for Python but delivers them through an accessible, browser-based interface

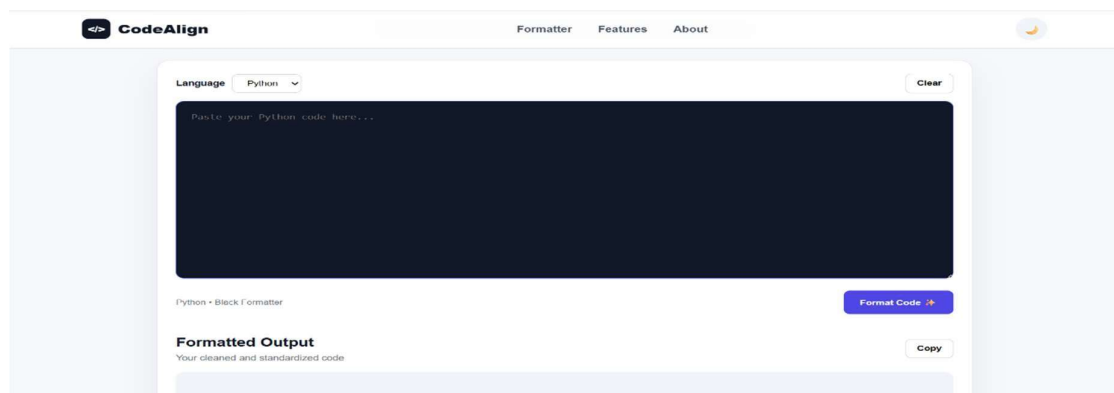
2.4 Research / Engineering Gap

Existing formatters are effective but are either tied to a specific editor/environment, require local installation and configuration, or hide the internal lexical/structural analysis from the user. There is a gap for a simple, accessible, browser-based system that both formats code and makes the underlying compiler-design pipeline (tokenization → structural analysis → rule application) visible and understandable — valuable for students and for developers who want a quick, no-install formatting utility.

2.5 Proposed Contribution

CodeAlign contributes a lightweight, web-based formatting tool that implements its own lexical analysis and structural analysis stages (rather than wrapping an existing formatter), served through a Flask back-end with an interactive front-end editor. This makes the compilerdesign pipeline directly demonstrable while still producing practical, Black-style formatted output, and it establishes an extensible foundation for future multi-language and AST-based enhancements.

Fig 2:INPUT ENTERING FIELD



CHAPTER CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Student developers / programmers	Quickly convert messy or inconsistently indented code into a clean, readable version without installing local tooling.
Course instructors / evaluators	A working demonstration that clearly shows lexical and structural analysis concepts applied to a real tool.
Project team (maintainers)	A codebase that is simple enough to extend with new languages and features in future iterations.

Functional & Non-Functional Requirements

Requirement	Measurable Target
Functional	Accept pasted or uploaded source code and return formatted output
Functional	Correctly identify keywords, identifiers, operators, literals, and delimiters
Functional	Correctly detect nested block boundaries and apply proportional indentation
Functional	Provide Format and Clear controls and a readable output/editor pane
Non-Functional	Formatted output must preserve the logical behaviour of the input code
Non-Functional	Interface must be responsive across common screen sizes
Non-Functional	System should return formatted output for typical inputs within a few seconds

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
-----------------	-------------	-----------------------------

Technical	Formatting must not alter program semantics	Formatting rules act only on whitespace, indentation, and layout — never on tokens that affect logic
Academic / Regulatory	Project must demonstrate Compiler Design course concepts	Explicit lexical analysis and structural analysis modules were implemented rather than calling an external formatter library
Usability	Interface must be usable without installation	Delivered as a browser-based web application (HTML/CSS/JS front-end, Flask back-end)

3.3 Design Specifications

Parameter	Target/Requirement	Unit	Priority
Formatting scope	Indentation, spacing, bracket alignment	—	High
Response time	Formatted output returned promptly for typical inputs	seconds	High
Logic preservation	100% — no change to program behaviour	%	Critical
Supported input language (current)	Python (initial target)	—	Medium
Formatting scope	Indentation, spacing, bracket alignment	—	High

3.4 Alternative Solutions & Evaluation

Alternative 1 – Manual formatting: developers manually apply indentation and spacing without tool support, relying entirely on individual discipline and code review.

Alternative 2 – Wrapping an existing formatter (e.g., Black/Clang-Format as a library or subprocess) behind a simple web front-end: leverages a mature, proven formatting engine but forfeits the opportunity to demonstrate custom lexical/structural analysis.

Alternative 3 (selected) – CodeAlign’s custom pipeline: a purpose-built lexical analysis and structural analysis engine, styled on Black conventions, served through a lightweight Flask back-end and browser-based editor.

CHAPTER

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Performance	3	Low (manual, no tooling)	High (mature, optimized)	Medium (custom, unoptimized)
Cost	3	Free, no tool needed	Free but requires local install/config	Free, browser-based, no install
Safety	2	Fully manual, transparent	Opaque internal logic	Exposes lexical/structural analysis stages
Sustainability	3	None	None (uses existing engine)	High — direct compiler-design demonstration
Reliability	2	N/A	High (mature plugin systems)	Medium (custom engine, extensible by design)

3.5 Selected Design & Detailed Design

Alternative 3 was selected because it is the only option that satisfies the academic/regulatory constraint in Section 3.2 (demonstrating Compiler Design concepts directly) while still meeting the accessibility requirement of a no-install, browser-based tool identified in Section 3.1. The evaluation matrix in Section 3.4 shows it trading a small amount of raw performance and maturity (compared with wrapping Black/Clang-Format) for substantially higher transparency and academic relevance, which are the priority criteria for this project.

System architecture — the system consists of six cooperating components:

- User Interface – browser-based editor, Format/Clear controls, and output display.
- Input Processing Module – receives and validates the submitted source code.
- Lexical Analysis Module – identifies tokens (keywords, identifiers, operators, literals, delimiters, symbols).
- Structural Analysis Module – determines relationships between tokens, block boundaries, and nesting depth.
- Formatting Engine – applies indentation, spacing, and block-alignment rules based on the structural analysis.
- Output Module – reconstructs and displays the formatted source code.

System workflow / data flow: Source Code Input → Input Validation → Token Identification → Structural Analysis → Formatting (indentation, line breaks, spacing, block alignment) → Output Generation.

Illustrative token identification (for the input “if (a > b) { max = a; }”): if = keyword; (= delimiter; a = identifier; > = operator; b = identifier;) = delimiter; { = block delimiter; max = identifier; = = assignment operator; a = identifier; ; = statement delimiter; } = block delimiter.

Formatting algorithm (simplified): read the source line by line → identify opening/closing delimiters → detect current nesting level → increase indentation after an opening block and decrease it before a closing block → apply spacing rules around operators and keywords → preserve literals and expressions unchanged → generate the formatted output.

Example — Input: `if(a>b){max=a;}else{max=b;}` → Output:

```
if (a > b) {    max = a; } else {    max = b;
}

```

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Formatting engine approach	Wrap an existing formatter library (e.g., call Black as a subprocess)	Faster to implement, proven correctness	Does not demonstrate custom lexical/structural analysis	Implement a custom lexical + structural analysis pipeline in Python, styled on Black conventions
Back-end framework	Django	More built-in features (ORM, admin)	Heavier than needed for a single-purpose formatting API	Flask — lightweight framework matching the project's singleendpoint formatting workflow
Interface complexity	Full multi-language IDE-style editor	More powerful, closer to production tools	Higher development time; distracts from the compiler-design objective	Simple editor with language selector, Format/Clear controls, and output pane

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Reduces repeated manual formatting effort across a program's lifetime, indirectly reducing developer time and rework on every future edit	Automated, rule-based reformatting rather than one-off manual fixes; a lightweight Flask service with low compute requirements	Adopted a lightweight back-end (Flask, not a heavier framework) to keep runtime resource use low

CHAPTER

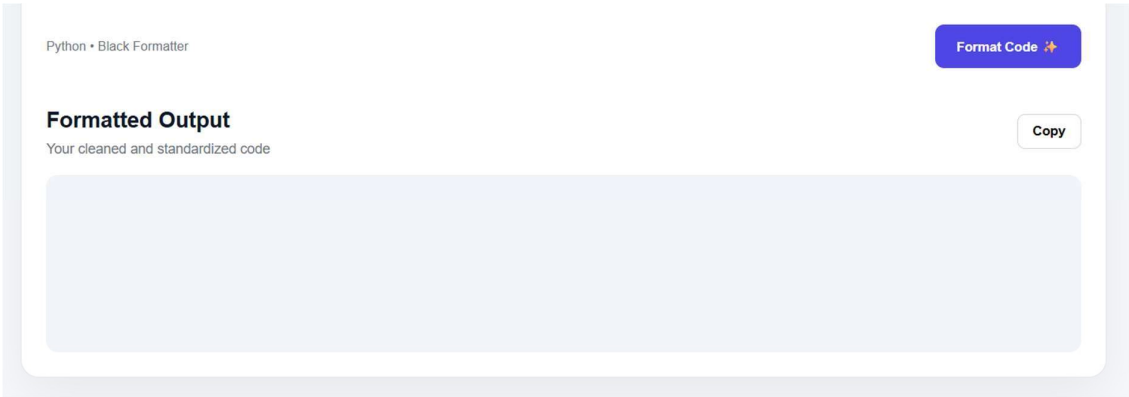
Ethics	Code submitted by users is processed only to produce formatted output; no external distribution of user code	No user code is stored beyond the request/response cycle	Formatting is applied without altering logic, preserving the original author's intended program behaviour
Health & Safety	Not applicable to a purely software/web-based tool with no physical hardware component	N/A	N/A
Security	Source code pasted by a user could contain sensitive logic or credentials	Input is processed transiently for formatting only, with no persistent storage or logging of submitted code	System designed to avoid retaining or transmitting user-submitted code beyond the formatting response

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Incorrect structural analysis on deeply nested/complex code alters formatting incorrectly	Medium	Medium	Medium	Test against a range of nested-block cases (TC02, TC03) before release	Low by mitigated test targeted cases
Formatting logic unintentionally changes program semantics	Low	High	Medium	Restrict formatting rules to whitespace/indentation only; never modify tokens	Low
Large input files cause slow response or timeout	Medium	Low	Low	Documented as a limitation; flagged for future performance optimization	Medium — accepted for current scope

Sensitive code pasted by a user is inadvertently retained	Low	Medium	Low	No persistent storage/logging of submitted code in the current implementation	Low
---	-----	--------	-----	---	-----

Fig 3: OUTPUT FIELD



CHAPTER CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project followed a staged development approach: problem identification, requirement analysis, system design, module-by-module development (lexical analysis, structural analysis, formatting engine, front-end), integration, testing, and documentation. The front-end and back-end were developed and tested independently before integration through a single HTTP-based formatting endpoint, which simplified debugging and allowed each subsystem in Section 3.5 to be validated in isolation.

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

Tool / Software / Equipment	Purpose	Stage Used
HTML	Structure of the CodeAlign web interface (editor, buttons, output area)	Front-end development
CSS	Layout, typography, spacing, responsive design, light/dark themes	Front-end development
JavaScript	Client-side interaction: handling user actions, theme switching, clearing the editor, updating the output area	Front-end development
Python	Back-end processing logic: token identification, structural analysis, formatting engine	Back-end development
Flask	Web framework connecting the front-end interface with the Pythonbased processing logic	Back-end development / integration

The front-end interface follows the flow Header → Code Editor → Control Buttons → Formatted Output → Additional Controls, and is built to remain usable across different screen sizes. The backend processing sequence is: User Input → HTTP Request → Flask Application → Source Code Processing → Formatting Logic → Formatted Result → HTTP Response → Web Interface. Separating front-end and back-end in this way improved the organization and maintainability of the application.

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion

Correct operator/expression spacing	Submit code with operators lacking spacing	Consistent spacing is applied around operators
Correct handling of multiple statements	Submit several statements in sequence	Statements are arranged on properly separated lines
Graceful handling of empty input	Submit an empty editor	System returns a suitable response without error
Interface theme switching	Toggle dark/light mode	Theme changes correctly across the interface

4.4 Results

Functional results: the system successfully accepts source code as input, identifies its lexical elements, analyzes brackets and nested structures, determines appropriate indentation levels, applies predefined formatting rules, generates properly aligned source code, and displays the formatted output through a responsive web interface with interactive controls. Example input/output — Input: `if(a>b){max=a;}else{max=b;}` Output: `if (a > b) { max = a; } else { max = b; }`

This confirms that the system improves indentation, spacing, and structural readability without changing the intended logic of the program.

4.5 Data Analysis & Engineering Judgment

Test Case	Expected Result	Actual Result	Status
TC01	Properly formatted statement	Successfully formatted	Pass
TC02	Correct indentation of conditional block	Successfully formatted	Pass
TC03	Multiple indentation levels for nested blocks	Successfully formatted	Pass
TC04	Appropriate operator/expression spacing	Successfully formatted	Pass
TC05	Proper line arrangement for multiple statements	Successfully formatted	Pass
TC06	Suitable handling of empty input	Handled by system	Pass
TC07	Correct dark-mode theme change	Successfully implemented	Pass

All seven test cases passed, indicating that the token-identification and structural-analysis stages correctly capture block boundaries across simple statements, conditional blocks, and nested structures, and that the formatting engine applies indentation and spacing deterministically from that structural information. No logic-altering defects were observed, which supports the design decision (Section 3.6) to restrict formatting operations strictly to whitespace and layout. The main current

CHAPTER

limitation, consistent with the risk register in Section 3.7, is untested performance on very large source files, since the test set used representative but modestly sized code samples rather than large production files.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement (Fully/Partially/Not Achieved)
Develop a web-based automatic source code formatting system	Working web application accepting pasted/uploaded code and returning formatted output	Fully Achieved
Analyze source code using compiler design concepts	Implemented lexical analysis and structural analysis modules (Section 3.5)	Fully Achieved
Identify keywords, identifiers, operators, literals, delimiters	Token table demonstrated for sample input (Section 3.5)	Fully Achieved
Understand nested blocks and program structures	TC02/TC03 nested-block indentation verified	Fully Achieved
Automatically apply indentation and formatting	Verified across TC01–TC05	Fully Achieved
Improve readability and consistency	Before/after formatting example (if/else) shows clear improvement	Fully Achieved
Provide a simple, user-friendly interface	Responsive editor with Format/Clear controls; TC07 confirms theme switching	Fully Achieved
Support multiple programming languages / AST-based analysis	Not implemented in current version; identified as future work	Not Achieved (Future Work)

4.7 Strengths & Limitations

Strengths

- Reduces manual formatting effort and improves source code readability.
- Provides consistent indentation and clear visualization of program structure.
- Simple, responsive, web-based interface requiring no installation.
- Directly demonstrates practical compiler design concepts (lexical and structural analysis).
- Modular architecture (Section 3.5) that can be extended to additional languages and features.

Limitations

- Currently supports formatting rules for a single target language (Python-style) rather than multiple languages.
- Does not build a full Abstract Syntax Tree, limiting handling of highly complex or ambiguous structures.
- No syntax error detection — malformed input may not be handled gracefully.

- Formatting rules are currently fixed and not user-customizable.
- Performance on very large source files has not been benchmarked.

4.8 Project Planning, Roles & Budget

Milestones

- Milestone 1 – Problem identification and requirement analysis.
- Milestone 2 – System design (architecture, workflow, module design).
- Milestone 3 – Front-end development (editor, controls, output display).
- Milestone 4 – Back-end development (lexical analysis, structural analysis, formatting engine, Flask integration).
- Milestone 5 – Integration and testing (TC01–TC07).
- Milestone 6 – Documentation and final report preparation.

ROLES & CONTRIBUTION & BUDGET:

Student	Assigned Responsibility	Actual Contribution
Arunprasath R	Front-end interface, lexical analysis module, UI-engine integration	Delivered editor UI, token identification logic, integration with formatting engine
Kailash V	Back-end development, structural analysis module, Flask integration, documentation	Delivered Flask back-end, structural analysis and indentation logic, project documentation
Item	Estimated Cost	Actual Cost
Hosting / deployment	Free-tier web hosting	Free-tier web hosting
Development tools & libraries	Open-source (HTML/CSS/JS, Python, Flask)	Open-source (no licence cost)

CHAPTER

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The CodeAlign – Automatic Source Code Formatter project was successfully designed and implemented as a web-based application for automatically analyzing and formatting source code. The project set out to reduce the effort involved in manual code formatting while improving the readability, consistency, and structure of programming code, and it demonstrates the practical application of Compiler Design concepts — lexical analysis, token identification, structural analysis, and syntax-oriented processing — in a real, deployable tool. By analyzing source code and applying predefined formatting rules, CodeAlign generates properly indented and organized code without affecting its intended logical structure, as confirmed by seven passing test cases covering statements, conditional blocks, nested structures, operator spacing, and interface behaviour. Development also provided practical, integrated experience across HTML, CSS, JavaScript, Python, and Flask, together with system design, debugging, testing, and user-interface development, bridging theoretical compiler design knowledge and hands-on software engineering practice.

5.2 Future Work

- Multi-language support (C, C++, Java, JavaScript, HTML, SQL) with language-specific lexical and syntax rules.
- Advanced syntax analysis using an Abstract Syntax Tree (AST) for more structure-aware formatting.
- Syntax error detection with meaningful error messages prior to formatting.
- Syntax highlighting to visually distinguish keywords, identifiers, operators, strings, and comments.
- Customizable formatting rules (indentation size, spaces vs. tabs, bracket style, line length).
- Code quality analysis with suggestions for common coding issues.

5.3 Professional Learning & Individual Reflection

Practical application of lexical analysis, tokenization, and syntax-oriented structural processing beyond traditional compiler construction. End-to-end web application development integrating

HTML, CSS, JavaScript, Python, and Flask. Systematic debugging approach: identify the problem → analyze the cause → develop a solution → test → verify the result.

Engineering & Professional Skills Developed

- Logical thinking while analyzing programming structures and designing formatting rules.
- Teamwork and task division across front-end and back-end responsibilities.
- Time management across the staged project plan (Section 4.8).
- Technical communication and documentation skills through report preparation and presentation.

Individual Reflection – Arunprasath R

The most difficult problem encountered was reliably identifying token boundaries and translating them into correct indentation for nested conditional blocks; this was resolved by testing incrementally against progressively more complex nested examples (TC02, TC03) before integrating with the back-end. A key decision made personally was to keep the front-end editor deliberately simple (a single editor pane with Format/Clear controls) rather than building a full multi-language IDE, based on the evaluation in Section 3.4 that this better matched the project's time budget and academic objective. Independently, I gained a much clearer practical understanding of how token identification feeds into structural analysis. If repeated, I would build the front-end with syntax highlighting from the start rather than as a later addition.

Individual Reflection – Kailash V

The most difficult problem was designing a structural analysis approach that correctly tracked nesting depth across multiple block types without a full parser; this was solved by implementing a bracket/indentation-tracking approach validated against the test cases in Section 4.3. A key engineering decision I made was choosing Flask over a heavier framework for the back-end, justified by the tradeoff analysis in Section 3.6 favouring a lightweight service matching the project's single-endpoint formatting workflow. I independently learned how to structure a formatting engine as a distinct, testable module separate from request handling. If repeated, I would design the back-end with AST-based analysis from the outset to make future language support easier to add.

CHAPTER REFERENCES

IEEE referencing style is used throughout.

- [1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Pearson Education.
- [2] K. D. Cooper and L. Torczon, *Engineering a Compiler*, 3rd ed. Morgan Kaufmann, 2022.
- [3] LLVM Project, “ClangFormat Documentation,” LLVM Documentation, 2026.
- [4] LLVM Project, “Clang-Format Style Options,” LLVM Documentation, 2026.
- [5] Prettier Team, “What is Prettier?,” Prettier Documentation, 2026.
- [6] Prettier Team, “Prettier API Documentation,” 2026.
- [7] Python Software Foundation, “Black – The Uncompromising Python Code Formatter,” 2026.
- [8] Python Software Foundation, *Python 3 Documentation*, 2025–2026.
- [9] Pallets Projects, *Flask Documentation*, 2025–2026.
- [10] Mozilla Developer Network (MDN), *HTML, CSS and JavaScript Documentation*, 2025–2026.
- [11] Mozilla Developer Network (MDN), “JavaScript Guide,” 2025–2026.
- [12] W3C, *HTML Living Standard*, World Wide Web Consortium, 2025–2026.
- [13] W3C, *CSS Specifications and Documentation*, World Wide Web Consortium, 2025–2026

APPENDICES

Appendix A – Detailed Calculations

Not applicable — this project is a software system with no numerical engineering calculations beyond the algorithmic logic described in Section 3.5.

Appendix B – Engineering Drawings / Schematics

System architecture, workflow, and algorithm flow are provided in Section 3.5 (Selected Design & Detailed Design).

Appendix C – Source Code / Code Repository Information

The formatting-engine token table and the if/else formatting example are included in Sections 3.5 and 4.4 as representative excerpts. Complete source code is maintained in the project repository/deployment referenced by the team and is available on request. Appendix D – Datasheets Not applicable.

Appendix E – Experimental / Raw Data

Full test-case results are provided in Section 4.3 (Test Plan & Verification) and Section 4.4 (Results).

Appendix F – Ethics / Institutional Approval

Not applicable — the project does not involve human subjects, animal subjects, or externally regulated data.

Appendix G – Project Meeting / Progress Records

Progress was tracked against the milestones listed in Section 4.8.

Appendix H – User/Industry Feedback

Not collected in the current phase of the project.

Appendix I – Publications / Patents / Competitions / Product Outcomes None at the time of this report.

Appendix J – Individual Contribution Evidence

See the Individual Contribution Statement and Section 4.8 (Roles & Contribution) of this report.

Mandatory for team projects.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)

Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)